

- - Voraussetzungen
 - Installation
- app/api/challenges/add/route
 - Functions
 - addChallenge()
 - POST()
- app/api/challenges/delete/route
 - Functions
 - DELETE()
 - deleteChallenge()
- app/api/challenges/list/route
 - Functions
 - GET()
 - getListOfChallenges()
- app/api/sandbox/add/route
 - Functions
 - POST()
- app/api/sandbox/list/route
 - Functions
 - GET()
 - getListOfSandboxProjects()
- app/api/sandbox/update/route
 - Functions
 - PATCH()
- app/api/users/create/route
 - Functions
 - addExpert()
 - addExpertFields()
 - addOrganization()
 - getOrganizationIdByName()
 - POST()
- app/api/users/delete/route
 - Functions
 - DELETE()
 - deleteExpert()
- app/api/users/list/route
 - Functions
 - GET()
 - getListOfPeopleWithOrganization()

- app/api/users/update/route
 - Functions
 - addOrganization()
 - getOrganizationIdByName()
 - PATCH()
 - updateExpert()
- app/challenge/add/page
 - Functions
 - default()
- app/challenge/page
 - Functions
 - default()
- app/components/Agentbar
 - Functions
 - Agentbar()
- app/components/Sidebar
 - Functions
 - Sidebar()
- app/components/Topbar
 - Functions
 - Topbar()
- app/layout
 - Variables
 - metadata
 - Functions
 - default()
- app/page
 - Functions
 - default()
- app/relations/Person
 - Interfaces
 - Expert
 - ExpertFormData
 - Organization
 - Variables
 - getUserFromDB
 - initialExperts
- app/relations/add/page
 - Functions
 - default()
- app/relations/page
 - Functions
 - default()

- app/relations/update/page
 - Functions
 - default()
- app/sandbox/add/page
 - Functions
 - default()
- app/sandbox/page
 - Functions
 - default()
- app/sandbox/project/details/page
 - Functions
 - default()
- app/search/page
 - Functions
 - default()
- lib/db
 - Variables
 - default
- lib/types
 - Classes
 - Challenge
 - ExpertFormData
 - NewProject
 - ProblemItem
 - ProjectDetails
 - Interfaces
 - AddExpertViewProps

Documentation

Eine Anwendung zur Verwaltung von Expertennetzwerken sowie Projekten und Challenges auf kommunaler Ebene. Zum Aufsetzen der Anwendung müssen folgende Schritte befolgt werden.

Voraussetzungen

Stelle sicher, dass du **Node.js** (v18.x oder höher) und einen Paketmanager (**npm**, **yarn** oder **pnpm**) installiert hast.

Installation

1. Repositorium klonen: ``bash git clone https://github.com/FischkopDev/heiProb
2. Dependencies installieren: npm install
3. Anwendung testen npm run test
4. Server starten npm run dev

Documentation

Documentation / [app/api/challenges/add/route](#)

app/api/challenges/add/route

Functions

addChallenge()

| **addChallenge**(body): Promise<number>

Defined in: [app/api/challenges/add/route.ts:21](#)

Fügt ein neues Problem (Challenge) in die Datenbank ein.

Parameters

body

Der Inhalt aus der JSON-Anfrage, der die Details des Problems enthält.

category

string

Die Kategorie des Problems.

description?

string

Eine detaillierte Beschreibung des Problems.

state

string

Der aktuelle Status/Zustand des Problems (z.B. "open", "closed").

title

string

Der Titel des Problems.

Returns

Promise<number>

Die ID des neu erstellten Problems als Zahl. Gibt 0 zurück, falls ein Fehler auftritt oder kein ID-Wert generiert wurde.

Throws

Wirft einen Fehler, wenn die Datenbankverbindung fehlschlägt oder die Abfrage ungültig ist. Der Fehler wird jedoch im catch-Block abgefangen und loggt, gibt aber 0 zurück.

POST()

```
POST(request): Promise<NextResponse<{ error: string; required: string[]; }> | NextResponse<{ status: number; success: boolean; warning: string; }> | NextResponse<{ problemId: number; status: number; success: boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Defined in: [app/api/challenges/add/route.ts:80](#)

HTTP POST-Handler zum Erstellen eines neuen Challenges.

Dieser Endpunkt validiert die eingehenden Daten, prüft auf erforderliche Felder und ruft die interne Funktion `addChallenge` auf, um den Datensatz zu speichern.

Parameters

request

Request

Das HTTP-Request-Objekt, das den JSON-Body enthält.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |  
NextResponse<{ status: number; success: boolean; warning: string; }> |  
NextResponse<{ problemId: number; status: number; success: boolean; }> |  
NextResponse<{ details: any; error: string; }>>
```

Ein `NextResponse`-Objekt mit folgendem Inhalt: - **200 OK**: Bei Erfolg.
{ success: true, status: 200 } - **400 Bad Request**: Wenn erforderliche Felder fehlen. { error: "Missing required fields", required: [...] } - **500 Internal Server Error**: Bei Datenbankfehlern oder unerwarteten Ausnahmen.
{ error: "Failed to add challenge", details: error.message }

Example

```
// Beispiel für einen erfolgreichen Request  
const response = await fetch('/api/challenges', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({  
    title: "Rekursive Fibonacci",  
    category: "Algorithms",  
    state: "active",  
    description: "Berechne die Fibonacci-Zahl..."  
  })  
});  
const data = await response.json();  
console.log(data); // { success: true, status: 200 }
```

Documentation

[Documentation](#) / `app/api/challenges/delete/route`

app/api/challenges/delete/route

Functions

DELETE()

```
DELETE(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ message: string; success: boolean; }>>
```

Defined in: [app/api/challenges/delete/route.ts:74](#)

HTTP DELETE-Handler zum Entfernen eines Challenges.

Dieser Endpunkt erwartet eine JSON mit der `challenge_id`. Er validiert die Eingabe, ruft die Löschfunktion auf und gibt entsprechende Statuscodes zurück (200, 400, 404, 500).

Parameters

request

Request

Das HTTP-Request-Objekt, das den JSON-Body mit der `challenge_id` enthalten muss.

Returns

Promise<NextResponse<{ error: string; }> | NextResponse<{ message: string; success: boolean; }>>

Ein NextResponse-Objekt mit folgendem Inhalt: - **200 OK:** Bei erfolgreicher Löschung. { success: true, message: "Challenge with ID X successfully deleted" } - **400 Bad Request:** Wenn `challenge_id` fehlt oder ungültig ist. { error: "Challenge ist nicht vorhanden" } - **404 Not Found:** Wenn die ID in der Datenbank nicht existiert. { error: "Challenge with ID X nicht gefunden" } - **500 Internal Server Error:** Bei Datenbankfehlern oder unerwarteten Ausnahmen. { error: "Failed to delete challenge", details: error.message }

Example

```
// Beispiel für einen erfolgreichen Request
const response = await fetch('/api/challenges', {
  method: 'DELETE',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ challenge_id: 123 })
});
const data = await response.json();
console.log(data); // { success: true, message: "Challenge with ID
123 successfully deleted" }
```

deleteChallenge()

deleteChallenge(id): Promise<boolean>

Defined in: [app/api/challenges/delete/route.ts:28](#)

Löscht ein Problem (Challenge) aus der Datenbank basierend auf seiner ID.

Diese Funktion führt einen SQL-DELETE-Befehl aus und überprüft, ob tatsächlich eine Zeile betroffen war. Sie wirft einen Fehler, wenn die Datenbankabfrage selbst fehlschlägt (z.B. Verbindungsproblem).

Parameters

id

number

Die eindeutige ID des Problems (problem_id), das gelöscht werden soll.

Returns

Promise<boolean>

true, wenn ein Datensatz mit der angegebenen ID gefunden und gelöscht wurde. false, wenn kein Datensatz mit dieser ID existierte (aber keine Datenbankfehler auftraten).

Throws

Wirft einen Fehler, wenn die Datenbankverbindung unterbrochen wird oder die SQL-Abfrage syntaktisch/logisch fehlschlägt. Dieser Fehler muss vom Aufrufer (z.B. dem API-Handler) abgefangen werden.

Example

```
const success = await deleteChallenge(123);
if (success) {
  console.log("Challenge gelöscht.");
} else {
  console.log("Challenge nicht gefunden.");
}
```

Documentation

Documentation / [app/api/challenges/list/route](#)

app/api/challenges/list/route

Functions

GET()

```
GET(): Promise<NextResponse<{ challenges: any[]; count: number;
success: boolean; }> | NextResponse<{ details: any; error: string; }
>>
```

Defined in: [app/api/challenges/list/route.ts:63](#)

HTTP GET-Handler zum Abrufen der Liste aller Challenges.

Dieser Endpunkt ruft die Funktion `getListOfChallenges` auf und gibt die Ergebnisse als JSON-Objekt zurück. Er enthält Metadaten über den Erfolg und die Anzahl der Ergebnisse.

Returns

```
Promise<NextResponse<{ challenges: any[]; count: number; success:
boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Ein NextResponse-Objekt mit folgendem Inhalt: - **200 OK**: Bei Erfolg.

```
json      {      "success": true,
"challenges": [ /* Array von Challenge-Objekten */ ],
"count": 50 /* Anzahl der zurückgegebenen Einträge */ }
```

- **500 Internal Server Error**: Bei Datenbankfehlern oder unerwarteten Ausnahmen. json { "error": "Failed to fetch challenges", "details": "Fehlermeldung der Datenbank" }

Example

```
// Beispiel für einen erfolgreichen Request
const response = await fetch('/api/challenges');
const data = await response.json();
console.log(data.challenges); // Array der Challenges
console.log(data.count); // Anzahl der Ergebnisse
```

getListOfChallenges()

| `getListOfChallenges(): Promise<any[]>`

Defined in: [app/api/challenges/list/route.ts:23](#)

Holt eine Liste aller verfügbaren Challenges (Probleme) aus der Datenbank.

Diese Funktion führt eine SQL-Abfrage aus, die alle Einträge aus der Tabelle "Problem" auswählt und nach Titel alphabetisch sortiert.

Returns

Promise<any[]>

Ein Array mit den Datenbankzeilen (Objekten), die die Challenges repräsentieren. Jede Zeile enthält mindestens die Spalten der "Problem"-Tabelle. Gibt ein leeres Array zurück, wenn keine Einträge vorhanden sind.

Throws

Wirft einen Fehler, wenn die Datenbankverbindung fehlschlägt oder die Abfrage syntaktisch ungültig ist. Der Fehler wird nicht abgefangen und muss vom Aufrufer (z.B. dem API-Handler) behandelt werden.

Example

```
const challenges = await getListOfChallenges();
console.log(`Found ${challenges.length} challenges.`);
console.log(challenges[0]?.title); // Titel des ersten Challenges
```

Documentation

[Documentation](#) / [app/api/sandbox/add/route](#)

app/api/sandbox/add/route

Functions

POST()

```
POST(request): Promise<NextResponse<{ error: string; required:
string[]; }> | NextResponse<{ projectId: any; success: boolean; }> |
NextResponse<{ details: any; error: string; }>>
```

Defined in: [app/api/sandbox/add/route.ts:123](#)

Handler für HTTP POST-Anfragen zum Erstellen eines neuen Projekts.

Validiert die Eingabedaten und delegiert die eigentliche Erstellung an `addProject()`.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |
NextResponse<{ projectId: any; success: boolean; }> | NextResponse<{
details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit Erfolgsmeldung und Projekt-ID oder einem Fehlerstatus.

Example

```
// Erwartetes Body-Format:
{
  "title": "Neues Projekt",
  "state": "active",
  "members": [{ "name": "Max Mustermann", "role": "Lead" }]
}
```

Documentation

[Documentation](#) / [app/api/sandbox/list/route](#)

app/api/sandbox/list/route

Functions

GET()

```
GET(): Promise<NextResponse<{ count: number; projects: any[];
  success: boolean; }> | NextResponse<{ details: any; error: string; }
>>
```

Defined in: [app/api/sandbox/list/route.ts:84](#)

Handler für HTTP GET-Anfragen zum Abrufen der Sandbox-Projektliste.

Delegiert die Datenbankabfrage an `getListOfSandboxProjects()` und gibt die Ergebnisse als JSON-Antwort zurück.

Returns

```
Promise<NextResponse<{ count: number; projects: any[]; success:
  boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit: - success: Boolean, ob die Anfrage erfolgreich war - projects: Array der Projekt-Objekte - count: Anzahl der zurückgegebenen Projekte

Throws

Gibt bei Fehlern eine JSON-Antwort mit Status 500 zurück und enthält Fehlerdetails im `details`-Feld.

Example

```
// Beispielantwort bei Erfolg:
{
  "success": true,
  "projects": [
    {
      "id": 1,
      "title": "Beispielprojekt",
      "experts": [
        { "expert_id": 5, "name": "Max Mustermann", "role":
"Lead" }
      ]
    }
  ],
  "count": 1
}
```

getListOfSandboxProjects()

`getListOfSandboxProjects(): Promise<any[]>`

Defined in: [app/api/sandbox/list/route.ts:21](#)

Ruft eine Liste von Sandbox-Projekten aus der Datenbank ab.

Diese Funktion führt eine komplexe Abfrage durch, die: 1. Alle Projekte aus der Tabelle "Project" selektiert 2. Über "ProjectRelation" mit Experten verknüpft (LEFT JOIN) 3. Experteninformationen als JSON-Array aggregiert 4. Nach dem letzten Update-Datum absteigend sortiert

Returns

`Promise<any[]>`

Ein Array von Projekt-Objekten, wobei jedes Projekt ein Feld `experts` enthält, das ein Array von Experten mit `expert_id`, `name` und `role` ist.

Remarks

- Verwendet COALESCE und FILTER um leere Experten-Arrays als [] zurückzugeben
- DISTINCT verhindert Duplikate bei Mehrfachverknüpfungen
- Die Abfrage ist auf 50 Ergebnisse limitiert für Performance

Documentation

[Documentation](#) / [app/api/sandbox/update/route](#)

app/api/sandbox/update/route

Functions

PATCH()

```
PATCH(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ projectId: any; success: boolean; }>>
```

Defined in: [app/api/sandbox/update/route.ts:186](#)

Handler für HTTP PATCH-Anfragen zum Aktualisieren eines bestehenden Projekts.

Validiert die Anwesenheit der `project_id`, ruft die Update-Funktion auf und gibt das Ergebnis als JSON zurück.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

Promise<NextResponse<{ error: string; }> | NextResponse<{ projectId: any; success: boolean; }>>

Ein JSON-Antwortobjekt: - 200: Erfolg mit projectId - 400: Fehlende project_id - 404: Projekt nicht gefunden - 500: Interner Serverfehler

Example

```
// Erwartetes Body-Format:
{
  "project_id": 123,
  "title": "Neuer Titel",
  "members": [{ "name": "Anna Schmidt", "role": "Developer" }]
}
```

Documentation

[Documentation](#) / [app/api/users/create/route](#)

app/api/users/create/route

Functions

addExpert()

```
addExpert(body): Promise<{ expertId: any; fields:
  QueryResult<any>[]; }>
```

Defined in: [app/api/users/create/route.ts:139](#)

Erstellt einen neuen Experten in der Datenbank.

Dieser Prozess beinhaltet: 1. Prüfung, ob die angegebene primary_organization existiert. 2. Falls nein: Automatische Erstellung der Organisation mit Standardwerten für field und description. 3. Einfügen des Experten mit der aufgelösten Organisations-ID. 4. Optional: Einfügen der Expertisefelder (expertFields).

Parameters

body

any

Das Objekt mit den Experten-Daten.

Returns

Promise<{ expertId: any; fields: QueryResult<any>[]; }>

Objekt mit expertId und fields (Array von Einfügeergebnissen).

Remarks

- Die Funktion erstellt automatisch eine Organisation, wenn sie nicht existiert, wobei field und description leer gesetzt werden.
 - Boolesche Felder (economic, science, social) defaults auf false, wenn nicht übergeben.
 - Expertisefelder werden nach dem Einfügen des Experten hinzugefügt.
-

addExpertFields()

```
addExpertFields(expertId, expertFields):  
Promise<QueryResult<any>[]>
```

Defined in: [app/api/users/create/route.ts:82](#)

Fügt Expertisefelder für einen Experten in die Datenbank ein.

Parameters

expertId

number

Die ID des Experten.

expertFields

string[]

Array von Feld-Strings, die die Expertisen des Experten darstellen.

Returns

Promise<QueryResult<any>[]>

Array der Datenbank-Einfügeergebnisse (pg.QueryResult[]).

Remarks

- Fängt Fehler ab und loggt sie, wirft aber keine Exception.
 - Gibt ein leeres Array zurück, wenn expertFields leer oder undefined ist.
-

addOrganization()

```
addOrganization(name, location, field, description):  
Promise<any>
```

Defined in: [app/api/users/create/route.ts:19](#)

Fügt eine neue Organisation in die Datenbank ein.

Parameters

name

string

Der Name der Organisation (erforderlich).

location

string

Der Standort der Organisation.

field

string

Das Tätigkeitsfeld der Organisation.

description

string

Eine Beschreibung der Organisation.

Returns

Promise<any>

Die neu generierte `organization_id` bei Erfolg. Gibt `0` zurück, wenn ein Fehler auftritt oder kein ID-Wert zurückgegeben wird.

Remarks

- Logs Fehler an die Konsole, wirft aber keinen Exception, um den Aufrufer nicht zu blockieren.
 - Die Rückgabe von `0` bei Fehlern erfordert eine explizite Prüfung durch den Aufrufer.
-

getOrganizationIdByName()

| `getOrganizationIdByName(name): Promise<number | null>`

Defined in: [app/api/users/create/route.ts:48](#)

Sucht die eindeutige ID einer Organisation anhand ihres Namens.

Parameters

name

string

Der Name der Organisation, nach dem gesucht werden soll.

Returns

Promise<number | null>

Die `organization_id` als Zahl, falls gefunden, sonst `null`.

Remarks

- Nutzt `LIMIT 1`, da von eindeutigen Namen ausgegangen wird.
 - Fängt Datenbankfehler ab und gibt `null` zurück.
-

POST()

```
POST(request): Promise<NextResponse<{ error: string; required: string[]; }> | NextResponse<{ expertId: any; fieldsAdded: number; success: boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Defined in: [app/api/users/create/route.ts:195](#)

Handler für HTTP POST-Anfragen zum Erstellen eines neuen Experten.

Validiert die erforderlichen Felder (name, prename, email) und delegiert die Erstellung an `addExpert()`.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |  
NextResponse<{ expertId: any; fieldsAdded: number; success: boolean; }> |  
NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt: - 200: Erfolg mit { success: true, expertId: number, fieldsAdded: number } - 400: Fehlende Pflichtfelder - 500: Interner Serverfehler

Example

```
// Erwartetes Body-Format:  
{  
  "name": "Max",  
  "prename": "Mustermann",  
  "email": "max@example.com",  
  "primary_organization": "Beispiel GmbH",  
  "title": "Dr.",  
  "location": "Berlin",  
  "economic": true,  
  "expertFields": ["KI", "Machine Learning", "Data Science"]  
}
```

app/api/users/delete/route

Functions

DELETE()

```
DELETE(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ message: string; success: boolean; }>>
```

Defined in: [app/api/users/delete/route.ts:75](#)

Handler für HTTP DELETE-Anfragen zum Entfernen eines Experten.

Extrahiert die `expert_id` aus dem Request-Body, validiert sie und ruft die Löschfunktion auf. Unterscheidet zwischen fehlender ID, nicht existierendem Experten und technischen Fehlern.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

Promise<NextResponse<{ error: string; }> | NextResponse<{ message: string; success: boolean; }>>

Ein JSON-Antwortobjekt: - 200: Erfolg mit Bestätigungsmeldung. - 400: Fehlende oder ungültige `expert_id` im Body. - 404: Kein Experte mit der angegebenen ID gefunden. - 500: Interner Serverfehler bei der Datenbankabfrage.

Example

```
// Erwarteter Request-Body:  
{ "expert_id": 123 }
```

Remarks

- Die Validierungsmeldungen sind teilweise auf Deutsch (“User ist nicht vorhanden”), während die Erfolgsnachrichten auf Englisch sind. Für Konsistenz könnte man die Sprache vereinheitlichen.
-

deleteExpert()

```
| deleteExpert(id): Promise<boolean>
```

Defined in: [app/api/users/delete/route.ts:19](#)

Löscht einen Experten aus der Datenbank basierend auf der ID.

Parameters

id

number

Die eindeutige ID des zu löschenden Experten (expert_id).

Returns

Promise<boolean>

true, wenn ein Datensatz erfolgreich gelöscht wurde. false, wenn kein Datensatz mit dieser ID existierte (und somit nichts gelöscht wurde).

Throws

Wirft den ursprünglichen Datenbankfehler, falls die Abfrage technisch fehlschlägt (z.B. Verbindungsfehler, Syntaxfehler), damit der Aufrufer dies behandeln kann.

Remarks

- Die Abfrage nutzt RETURNING expert_id, um zu prüfen, ob tatsächlich eine Zeile betroffen war.

- Ein false-Rückgabewert bedeutet nicht zwangsläufig einen Fehler, sondern dass die ID nicht existierte.

Documentation

[Documentation](#) / [app/api/users/list/route](#)

app/api/users/list/route

Functions

GET()

```
GET(): Promise<NextResponse<{ count: number; experts: any[];
  success: boolean; }> | NextResponse<{ details: any; error: string; }
>>
```

Defined in: [app/api/users/list/route.ts:82](#)

Handler für HTTP GET-Anfragen zum Abrufen der Liste aller Personen mit Organisationsdaten und Expertisefeldern.

Delegiert die Datenbankabfrage an `getListOfPeopleWithOrganization()` und formatiert die Antwort als JSON mit Erfolgsmeldung, Ergebnisliste und Gesamtanzahl.

Returns

```
Promise<NextResponse<{ count: number; experts: any[]; success: boolean; }
> | NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit: - success: Boolean, ob die Anfrage erfolgreich war - experts: Array der Experten-Objekte (inkl. Organisations- und Expertisefelder-Daten) - count: Anzahl der zurückgegebenen Experten

Throws

Gibt bei Fehlern eine JSON-Antwort mit Status 500 zurück und enthält Fehlerdetails im details-Feld.

Example

```
// Beispielantwort:
{
  "success": true,
  "experts": [
    {
      "expert_id": 1,
      "name": "Anna",
      "prename": "Schmidt",
      "organization": {
        "organization_id": 5,
        "name": "TechCorp",
        "location": "Berlin",
        "field": "IT",
        "description": "Ein Technologieunternehmen"
      },
      "expertFields": ["Künstliche Intelligenz", "Machine Learning", "Data Science"]
    }
  ],
  "count": 1
}
```

getListOfPeopleWithOrganization()

| **getListOfPeopleWithOrganization():** Promise<any[]>

Defined in: [app/api/users/list/route.ts:21](#)

Ruft eine Liste von Experten aus der Datenbank ab, jeweils ergänzt um ihre zugehörige Organisation und Expertisefelder.

Diese Funktion führt JOINS zwischen den Tabellen "Expert", "Organization" und "ExpertField" durch, um die Organisationsdaten und Expertisefelder als verschachtelte JSON-Objekte zurückzugeben.

Returns

Promise<any[]>

Ein Array von Objekten, wobei jedes Objekt alle Felder des Experten (e. *) sowie folgende Felder enthält: - organization: JSON-Objekt mit Organisations-Daten (organization_id, name, location, field, description) - expertFields: JSON-Array der Expertisefelder des Experten

Remarks

- Die Ergebnisse sind alphabetisch nach dem Vornamen (e.name) sortiert.
- Die Abfrage ist auf maximal 50 Ergebnisse begrenzt (LIMIT 50).
- Nur Experten mit einer gültigen Zuordnung zu einer Organisation werden zurückgegeben (INNER JOIN).
- Expertisefelder werden mittels LEFT JOIN und Aggregation geladen, Experten ohne Felder erhalten ein leeres Array.

Documentation

Documentation / [app/api/users/update/route](#)

app/api/users/update/route

Functions

addOrganization()

```
addOrganization(name, location, field, description):  
  Promise<any>
```

Defined in: [app/api/users/update/route.ts:19](#)

Fügt eine neue Organisation in die Datenbank ein.

Parameters

name

string

Der Name der Organisation (erforderlich).

location

string

Der Standort der Organisation.

field

string

Das Tätigkeitsfeld der Organisation.

description

string

Eine Beschreibung der Organisation.

Returns

Promise<any>

Die neu generierte `organization_id` bei Erfolg. Gibt `0` zurück, wenn ein Fehler auftritt oder kein ID-Wert zurückgegeben wird.

Remarks

- Logs Fehler an die Konsole, wirft aber keine Exception, um den Aufrufer nicht zu blockieren.
 - Die Rückgabe von `0` bei Fehlern erfordert eine explizite Prüfung durch den Aufrufer.
-

getOrganizationIdByName()

```
| getOrganizationIdByName(name): Promise<number | null>
```

Defined in: [app/api/users/update/route.ts:48](#)

Sucht die eindeutige ID einer Organisation anhand ihres Namens.

Parameters

name

string

Der Name der Organisation, nach dem gesucht werden soll.

Returns

Promise<number | null>

Die organization_id als Zahl, falls gefunden, sonst null.

Remarks

- Nutzt LIMIT 1, da von eindeutigen Namen ausgegangen wird.
 - Fängt Datenbankfehler ab und gibt null zurück.
-

PATCH()

```
PATCH(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ data: any; success: boolean; }>>
```

Defined in: [app/api/users/update/route.ts:173](#)

Handler für HTTP PATCH-Anfragen zum Aktualisieren eines bestehenden Experten.

Extrahiert die expert_id aus dem Request-Body, trennt sie von den Update-Daten und delegiert die Aktualisierung an updateExpert().

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

Promise<NextResponse<{ error: string; }> | NextResponse<{ data: any; success: boolean; }>>

Ein JSON-Antwortobjekt: - 200: Erfolg mit dem aktualisierten Experten-Objekt in data. - 400: Fehlende expert_id. - 404: Kein Experte mit der angegebenen ID gefunden. - 500: Interner Serverfehler bei der Datenbankabfrage.

Example

```
// Erwarteter Request-Body:
{
  "expert_id": 123,
  "title": "Dr.",
  "primary_organization": "Neue Firma AG"
}
```

Remarks

- Nur die im Body enthaltenen Felder werden aktualisiert; alle anderen bleiben unverändert.
 - Die Fehlermeldung bei fehlender ID ist auf Englisch, während andere Teile des Codes teils Deutsch verwenden (Konsistenz könnte verbessert werden).
-

updateExpert()

| **updateExpert**(id, body): Promise<any>

Defined in: [app/api/users/update/route.ts:99](#)

Aktualisiert die Daten eines Experten in der Datenbank.

Dieser Prozess beinhaltet: 1. Prüfung und ggf. automatische Erstellung der angegebenen primary_organization. 2. Aktualisierung der Expertenfelder unter Verwendung von COALESCE, um nicht übergebene Werte beizubehalten.

Parameters

id

number

Die ID des zu aktualisierenden Experten (expert_id).

body

any

Das Objekt mit den zu aktualisierenden Daten.

Returns

Promise<any>

Das aktualisierte Experten-Objekt (inkl. aller Felder) bei Erfolg. Gibt undefined zurück, wenn kein Experte mit der angegebenen ID existiert.

Remarks

- Wenn `primary_organization` angegeben ist, wird geprüft, ob sie existiert. Falls nicht, wird sie mit dem aktuellen `location` (falls vorhanden) und leeren Feldern für `field/description` erstellt.
- Die SQL-Abfrage nutzt `COALESCE($1, column)`, um nur übermittelte Werte zu überschreiben.
- Falls `primary_organization` nicht im Body ist, bleibt die bisherige Zuordnung erhalten.

Documentation

Documentation / `app/challenge/add/page`

app/challenge/add/page

Functions

default()

| **default()**: Element

Defined in: `app/challenge/add/page.tsx:39`

Eine Next.js-Client-Komponente, die ein Formular zum Erstellen einer neuen Herausforderung (Challenge) bereitstellt. *

Returns

Element

Ein gerendertes React-Formularlement.

Documentation

[Documentation](#) / [app/challenge/page](#)

app/challenge/page

Functions

default()

| **default()**: Element

Defined in: [app/challenge/page.tsx:19](#)

Eine Next.js-Client-Komponente, die eine Übersicht aller vorhandenen Probleme darstellt, Daten live von der API lädt und das Löschen sowie die Detailansicht verwaltet. *

Returns

Element

Ein gerendertes UI-Element für die Problem-Verwaltung.

Documentation

[Documentation](#) / [app/components/Agentbar](#)

app/components/Agentbar

Functions

Agentbar()

| **Agentbar()**: Element

Defined in: [app/components/Agentbar.tsx:10](#)

Die seitliche Ansicht auf dem der KI-Agent platziert wird. Hier sollen die Befehle an den Agent übernommen werden.

Returns

Element

Documentation

[Documentation](#) / [app/components/Sidebar](#)

app/components/Sidebar

Functions

Sidebar()

| **Sidebar()**: Element

Defined in: [app/components/Sidebar.tsx:12](#)

Die Navigationsleiste mit Verbindung zu den Bereichen Expertennetzwerk, Challenges und Sandbox-Projekte.

Returns

Element

Documentation

[Documentation](#) / [app/components/Topbar](#)

app/components/Topbar

Functions

Topbar()

| **Topbar()**: Element

Defined in: [app/components/Topbar.tsx:14](#)

Die Topbar, die auf allen Seiten sichtbar ist. Sie enthält eine Suchleiste für die schnelle Suche nach Expertisen, Problemen oder Projekten. Außerdem gibt es zwei prominente Buttons: Einen zum Hinzufügen eines neuen Problems (Challenge) und einen zum Hinzufügen eines neuen Projekts (Sandbox).

Returns

Element

Documentation

[Documentation](#) / [app/layout](#)

app/layout

Variables

metadata

| `const metadata:` Metadata

Defined in: [app/layout.tsx:11](#)

Functions

default()

| **default**(__namedParameters): Element

Defined in: [app/layout.tsx:18](#)

Parameters

__namedParameters

children

ReactNode

Returns

Element

Documentation

[Documentation](#) / [app/page](#)

app/page

Functions

default()

| **default**(): Element

Defined in: [app/page.tsx:5](#)

Returns

Element

app/relations/Person

Interfaces

Expert

Defined in: app/relations/Person.tsx:12

Properties

description?

| optional **description?**: string

Defined in: app/relations/Person.tsx:22

economic

| **economic**: boolean

Defined in: app/relations/Person.tsx:25

email?

| optional **email?**: string

Defined in: app/relations/Person.tsx:20

expert_fields

| **expert_fields**: string[]

Defined in: app/relations/Person.tsx:24

expert_id

| **expert_id**: number

Defined in: [app/relations/Person.tsx:13](#)

last_contact?

| optional **last_contact?**: string

Defined in: [app/relations/Person.tsx:23](#)

name

| **name**: string

Defined in: [app/relations/Person.tsx:14](#)

organization?

| optional **organization?**: [Organization](#)

Defined in: [app/relations/Person.tsx:28](#)

other_organizations

| **other_organizations**: string[]

Defined in: [app/relations/Person.tsx:18](#)

phone?

| optional **phone?**: string

Defined in: [app/relations/Person.tsx:21](#)

prename

| **prename**: string

Defined in: [app/relations/Person.tsx:15](#)

primary_organization

| **primary_organization:** string

Defined in: [app/relations/Person.tsx:17](#)

science

| **science:** boolean

Defined in: [app/relations/Person.tsx:26](#)

scientificAreas

| **scientificAreas:** string[]

Defined in: [app/relations/Person.tsx:19](#)

social

| **social:** boolean

Defined in: [app/relations/Person.tsx:27](#)

title?

| optional **title?:** string

Defined in: [app/relations/Person.tsx:16](#)

ExpertFormData

Defined in: [app/relations/Person.tsx:31](#)

Properties

description

| **description:** string

Defined in: [app/relations/Person.tsx:41](#)

economic

| **economic:** boolean

Defined in: [app/relations/Person.tsx:43](#)

email

| **email:** string

Defined in: [app/relations/Person.tsx:38](#)

expert_fields

| **expert_fields:** string

Defined in: [app/relations/Person.tsx:42](#)

last_contact?

| optional **last_contact?:** string

Defined in: [app/relations/Person.tsx:40](#)

name

| **name:** string

Defined in: [app/relations/Person.tsx:32](#)

other_organizations

| **other_organizations:** string

Defined in: [app/relations/Person.tsx:36](#)

phone

| **phone:** string

Defined in: [app/relations/Person.tsx:39](#)

prename

| **prename:** string

Defined in: [app/relations/Person.tsx:34](#)

primary_organization

| **primary_organization:** string

Defined in: [app/relations/Person.tsx:35](#)

science

| **science:** boolean

Defined in: [app/relations/Person.tsx:44](#)

scientificAreas

| **scientificAreas:** string

Defined in: [app/relations/Person.tsx:37](#)

social

| **social:** boolean

Defined in: [app/relations/Person.tsx:45](#)

title

| **title:** string

Defined in: [app/relations/Person.tsx:33](#)

Organization

Defined in: [app/relations/Person.tsx:3](#)

Properties

description?

| optional **description?**: string

Defined in: [app/relations/Person.tsx:8](#)

field?

| optional **field?**: string

Defined in: [app/relations/Person.tsx:7](#)

location?

| optional **location?**: string

Defined in: [app/relations/Person.tsx:6](#)

name

| **name**: string

Defined in: [app/relations/Person.tsx:5](#)

organization_id

| **organization_id**: number

Defined in: [app/relations/Person.tsx:4](#)

Variables

getUserFromDB

| const **getUserFromDB**: () => Promise<[Expert\[\]](#)>

Defined in: [app/relations/Person.tsx:48](#)

Returns

Promise<[Expert\[\]](#)>

initialExperts

`const initialExperts: Expert[]`

Defined in: [app/rerelations/Person.tsx:85](#)

Documentation

[Documentation](#) / [app/rerelations/add/page](#)

app/rerelations/add/page

Functions

default()

`default(props): Element`

Defined in: [app/rerelations/add/page.tsx:67](#)

Eine Next.js-Client-Komponente, die ein Formular zum Erstellen und Speichern eines neuen Experten-Profiles bereitstellt. Die Daten werden sowohl an die Datenbank übertragen als auch über Callbacks an die übergeordnete Komponente zurückgegeben. *

Parameters

props

[AddExpertViewProps](#)

Die Props für die Komponente.

Returns

Element

Ein gerendertes UI-Formular für die Expertenerfassung.

Documentation

[Documentation](#) / [app/relations/page](#)

app/relations/page

Functions

default()

| **default()**: Element

Defined in: [app/relations/page.tsx:34](#)

Returns

Element

Documentation

[Documentation](#) / [app/relations/update/page](#)

app/relations/update/page

Functions

default()

| **default(props)**: Element

Defined in: [app/relations/update/page.tsx:67](#)

Eine Next.js-Client-Komponente, die als Formular zum Bearbeiten oder Erstellen eines Experten-Profiles dient. Wenn ein expertId-Query-Parameter in der URL existiert, schaltet die Ansicht in den **Bearbeitungsmodus** und lädt die bestehenden Daten vorab aus der Datenbank. *

Parameters

props

`AddExpertViewProps`

Die Props für die Komponente.

Returns

`Element`

Ein gerendertes UI-Formular zur Expert*innen-Verwaltung.

Documentation

[Documentation](#) / `app/sandbox/add/page`

app/sandbox/add/page

Functions

default()

| **default()**: `Element`

Defined in: [app/sandbox/add/page.tsx:50](#)

Eine Next.js-Client-Komponente, die eine Eingabemaske zum Erstellen neuer Projekte bietet. Ermöglicht die Pflege von Metadaten sowie die dynamische Zuordnung von Experten über ein RACI-Rollenmodell. *

Returns

`Element`

Ein gerendertes Formular zur Projekterstellung.

Documentation

app/sandbox/page

Functions

default()

| **default()**: Element

Defined in: [app/sandbox/page.tsx:21](#)

Eine Next.js-Client-Komponente, die eine tabellarische Übersicht oder Liste aller Sandbox-Projekte aus der Datenbank anzeigt. Sie lädt die Projektdaten asynchron und bereitet diese direkt für die Anzeige auf. *

Returns

Element

Die gerenderte Übersicht der Sandbox-Projekte.

Documentation

[Documentation](#) / [app/sandbox/project/details/page](#)

app/sandbox/project/details/page

Functions

default()

| **default()**: Element

Defined in: [app/sandbox/project/details/page.tsx:61](#)

Eine Next.js-Client-Komponente, die eine Detailansicht zur Bearbeitung eines bestehenden Projekts bereitstellt. Lädt Projektdaten sowie die globale Expertenliste parallel, erlaubt die Modifikation aller Felder inklusive RACI-Teamzuweisungen (mit Duplikatsprüfung) und speichert die Änderungen via PATCH-Request. *

Returns

Element

Die gerenderte Detail- und Bearbeitungsseite des Projekts.

Documentation

[Documentation](#) / [app/search/page](#)

app/search/page

Functions

default()

| **default()**: Element

Defined in: [app/search/page.tsx:38](#)

Eine Next.js-Client-Komponente, die eine globale, interaktive Suchoberfläche bereitstellt. Durchsucht parallel Challenges, Projekte und Expertinnen *anhand einer einzigen Benutzereingabe (Query) in Echtzeit auf Client-Eite.*

Returns

Element

Das gerenderte Suchfenster inklusive Suchergebnissen.

Documentation

[Documentation](#) / [lib/db](#)

lib/db

Variables

default

```
| const default: Pool
```

Defined in: [lib/db.ts:3](#)

Documentation

[Documentation](#) / lib/types

lib/types

Classes

Challenge

Defined in: [lib/types.ts:13](#)

Repräsentiert eine städtische oder organisatorische Herausforderung (Challenge). *mermaid classDiagram class Challenge { +string id +string title +string department +string status }

Constructors

Constructor

```
| new Challenge(id, title, department, status): Challenge
```

Defined in: [lib/types.ts:23](#)

Parameters

id

string

title

string

department

string

status

string

Returns

Challenge

Properties

_department

```
| private _department: string
```

Defined in: [lib/types.ts:19](#)

Das zuständige Amt oder die Abteilung.

_id

```
| private _id: string
```

Defined in: [lib/types.ts:15](#)

Eindeutige ID der Challenge.

_status

| private **_status**: string

Defined in: [lib/types.ts:21](#)

Der aktuelle Status der Challenge (z. B. “Aktiv”, “In Vorbereitung”).

_title

| private **_title**: string

Defined in: [lib/types.ts:17](#)

Der Titel oder die Überschrift der Challenge.

Accessors

department

Get Signature

| **get department**(): string

Defined in: [lib/types.ts:46](#)

Returns

string

Set Signature

| **set department**(value): void

Defined in: [lib/types.ts:50](#)

Parameters

value

string

Returns

void

id

Get Signature

| **get id():** string

Defined in: [lib/types.ts:30](#)

Returns

string

Set Signature

| **set id(value):** void

Defined in: [lib/types.ts:34](#)

Parameters

value

string

Returns

void

status

Get Signature

| **get status():** string

Defined in: [lib/types.ts:54](#)

Returns

string

Set Signature

| **set status**(value): void

Defined in: [lib/types.ts:58](#)

Parameters

value

string

Returns

void

title

Get Signature

| **get title**(): string

Defined in: [lib/types.ts:38](#)

Returns

string

Set Signature

| **set title**(value): void

Defined in: [lib/types.ts:42](#)

Parameters

value

string

Returns

void

Expert

Defined in: [lib/types.ts:181](#)

Repräsentiert ein Profil einer Expertin *oder einer Ansprechperson*.

```
mermaid classDiagram class Expert { +string id +string name +string role +string[] skills }
```

Constructors

Constructor

`new Expert(id, name, role, skills): Expert`

Defined in: [lib/types.ts:191](#)

Parameters

id

string

name

string

role

string

skills

string[]

Returns

Expert

Properties

_id

```
| private _id: string
```

Defined in: [lib/types.ts:183](#)

Eindeutige ID der Expert*in.

_name

```
| private _name: string
```

Defined in: [lib/types.ts:185](#)

Der vollständige Name.

_role

```
| private _role: string
```

Defined in: [lib/types.ts:187](#)

Die fachliche Rolle oder Kernkompetenz.

_skills

```
| private _skills: string[]
```

Defined in: [lib/types.ts:189](#)

Eine Liste von spezifischen Fähigkeiten oder Schlagworten (Skills).

Accessors

id

Get Signature

| **get id():** string

Defined in: [lib/types.ts:198](#)

Returns

string

Set Signature

| **set id(value):** void

Defined in: [lib/types.ts:202](#)

Parameters

value

string

Returns

void

name

Get Signature

| **get name():** string

Defined in: [lib/types.ts:206](#)

Returns

string

Set Signature

| **set name**(value): void

Defined in: [lib/types.ts:210](#)

Parameters

value

string

Returns

void

role

Get Signature

| **get role**(): string

Defined in: [lib/types.ts:214](#)

Returns

string

Set Signature

| **set role**(value): void

Defined in: [lib/types.ts:218](#)

Parameters

value

string

Returns

void

skills

Get Signature

| **get skills**(): string[]

Defined in: [lib/types.ts:222](#)

Returns

string[]

Set Signature

| **set skills**(value): void

Defined in: [lib/types.ts:226](#)

Parameters

value

string[]

Returns

void

ExpertFormData

Defined in: [lib/types.ts:848](#)

Formulardaten-Struktur für Expertinnen.

```
mermaid classDiagram class ExpertFormData { +string name +string prename +string title +string primary_organization +string other_organizations +string scientificAreas +string email +string phone +string last_contact +string description +string expert_fields +boolean economic +boolean science +boolean social }
```

Constructors

Constructor

```
new ExpertFormData(name, prename, title, primary_organization, other_organizations, scientificAreas, email, phone, description, expert_fields, economic, science, social, last_contact?): ExpertFormData
```

Defined in: [lib/types.ts:878](#)

Parameters

name

string

prename

string

title

string

primary_organization

string

other_organizations

string

scientificAreas

string

email

string

phone

string

description

string

expert_fields

string

economic

boolean

science

boolean

social

boolean

last_contact?

string

Returns

ExpertFormData

Properties

_description

| private **_description**: string

Defined in: [lib/types.ts:868](#)

Eine detaillierte Beschreibung oder Kurzbiografie des Experten.

_economic

| private **_economic**: boolean

Defined in: [lib/types.ts:872](#)

Indikator, ob der Experte im wirtschaftlichen Bereich tätig ist.

_email

| private **_email**: string

Defined in: [lib/types.ts:862](#)

Die E-Mail-Adresse des Experten.

_expert_fields

| private **_expert_fields**: string

Defined in: [lib/types.ts:870](#)

Spezifische Fachbereiche oder Felder, in denen der Experte tätig ist.

_last_contact?

| private optional **_last_contact?**: string

Defined in: [lib/types.ts:866](#)

Datum des letzten Kontakts (optional).

_name

| private **_name**: string

Defined in: [lib/types.ts:850](#)

Der Nachname des Experten.

_other_organizations

| private **_other_organizations**: string

Defined in: [lib/types.ts:858](#)

Weitere Zugehörigkeiten oder Organisationen (kommagetrennt).

_phone

| private **_phone**: string

Defined in: [lib/types.ts:864](#)

Die Telefonnummer des Experten.

_prename

| private **_prename**: string

Defined in: [lib/types.ts:852](#)

Der Vorname des Experten.

_primary_organization

| private **_primary_organization**: string

Defined in: [lib/types.ts:856](#)

Die primäre Zugehörigkeit oder Organisation.

_science

| private **_science**: boolean

Defined in: [lib/types.ts:874](#)

Indikator, ob der Experte im wissenschaftlichen Bereich tätig ist.

_scientificAreas

| private **_scientificAreas**: string

Defined in: [lib/types.ts:860](#)

Wissenschaftliche oder fachliche Arbeitsbereiche.

_social

| private **_social**: boolean

Defined in: [lib/types.ts:876](#)

Indikator, ob der Experte im sozialen Bereich tätig ist.

_title

| private **_title**: string

Defined in: [lib/types.ts:854](#)

Der Titel oder akademische Grad (z. B. "Prof. Dr.").

Accessors

description

Get Signature

| **get description**(): string

Defined in: [lib/types.ts:982](#)

Returns

string

Set Signature

| **set description**(value): void

Defined in: [lib/types.ts:986](#)

Parameters

value

string

Returns

void

economic

Get Signature

| **get economic**(): boolean

Defined in: [lib/types.ts:998](#)

Returns

boolean

Set Signature

| **set economic**(value): void

Defined in: [lib/types.ts:1002](#)

Parameters

value

boolean

Returns

void

email

Get Signature

| **get email**(): string

Defined in: [lib/types.ts:958](#)

Returns

string

Set Signature

| **set email**(value): void

Defined in: [lib/types.ts:962](#)

Parameters

value

string

Returns

void

expert_fields

Get Signature

| **get expert_fields():** string

Defined in: [lib/types.ts:990](#)

Returns

string

Set Signature

| **set expert_fields(value):** void

Defined in: [lib/types.ts:994](#)

Parameters

value

string

Returns

void

last_contact

Get Signature

| **get last_contact():** string | undefined

Defined in: [lib/types.ts:974](#)

Returns

string | undefined

Set Signature

| **set last_contact**(value): void

Defined in: [lib/types.ts:978](#)

Parameters

value

string | undefined

Returns

void

name

Get Signature

| **get name**(): string

Defined in: [lib/types.ts:910](#)

Returns

string

Set Signature

| **set name**(value): void

Defined in: [lib/types.ts:914](#)

Parameters

value

string

Returns

void

other_organizations

Get Signature

```
| get other_organizations(): string
```

Defined in: [lib/types.ts:942](#)

Returns

string

Set Signature

```
| set other_organizations(value): void
```

Defined in: [lib/types.ts:946](#)

Parameters

value

string

Returns

void

phone

Get Signature

| **get phone():** string

Defined in: [lib/types.ts:966](#)

Returns

string

Set Signature

| **set phone(value):** void

Defined in: [lib/types.ts:970](#)

Parameters

value

string

Returns

void

prename

Get Signature

| **get prename():** string

Defined in: [lib/types.ts:918](#)

Returns

string

Set Signature

| **set prename**(value): void

Defined in: [lib/types.ts:922](#)

Parameters

value

string

Returns

void

primary_organization

Get Signature

| **get primary_organization**(): string

Defined in: [lib/types.ts:934](#)

Returns

string

Set Signature

| **set primary_organization**(value): void

Defined in: [lib/types.ts:938](#)

Parameters

value

string

Returns

void

science

Get Signature

| **get science()**: boolean

Defined in: [lib/types.ts:1006](#)

Returns

boolean

Set Signature

| **set science(value)**: void

Defined in: [lib/types.ts:1010](#)

Parameters

value

boolean

Returns

void

scientificAreas

Get Signature

| **get scientificAreas():** string

Defined in: [lib/types.ts:950](#)

Returns

string

Set Signature

| **set scientificAreas(value):** void

Defined in: [lib/types.ts:954](#)

Parameters

value

string

Returns

void

social

Get Signature

| **get social():** boolean

Defined in: [lib/types.ts:1014](#)

Returns

boolean

Set Signature

| **set social**(value): void

Defined in: [lib/types.ts:1018](#)

Parameters

value

boolean

Returns

void

title

Get Signature

| **get title**(): string

Defined in: [lib/types.ts:926](#)

Returns

string

Set Signature

| **set title**(value): void

Defined in: [lib/types.ts:930](#)

Parameters

value

string

Returns

void

ExpertOption

Defined in: [lib/types.ts:479](#)

Struktur für die Auswahlliste (Dropdown) der verfügbaren Experten. *

```
mermaid classDiagram class ExpertOption { +id: number | string +string name }
```

Constructors

Constructor

new ExpertOption(id, name): [ExpertOption](#)

Defined in: [lib/types.ts:485](#)

Parameters

id

string | number

name

string

Returns

ExpertOption

Properties

_id

| private **_id**: string | number

Defined in: lib/types.ts:481

Die ID des Experten (as number or string).

_name

| private **_name**: string

Defined in: lib/types.ts:483

Der vollständige Name des Experten.

Accessors

id

Get Signature

| **get id**(): string | number

Defined in: lib/types.ts:490

Returns

string | number

Set Signature

| **set id**(value): void

Defined in: lib/types.ts:494

Parameters

value

string | number

Returns

void

name

Get Signature

| **get name**(): string

Defined in: [lib/types.ts:498](#)

Returns

string

Set Signature

| **set name**(value): void

Defined in: [lib/types.ts:502](#)

Parameters

value

string

Returns

void

NewProject

Defined in: [lib/types.ts:526](#)

Struktur für das State-Objekt eines neu anzulegenden Projekts. *

```
mermaid classDiagram direction LR NewProject "1" --* "many" ProjectMember : enthält class NewProject { +string title +string description +string startDate +string endDate +string state +string location +string websiteUrl +string details +ProjectMember[] members }
```

Constructors

Constructor

```
new NewProject(title, description, startDate, endDate, state, location, websiteUrl, details, members): NewProject
```

Defined in: [lib/types.ts:546](#)

Parameters

title

string

description

string

startDate

string

endDate

string

state

string

location

string

websiteUrl

string

details

string

members

ProjectMember[]

Returns

NewProject

Properties

_description

| private ***_description***: string

Defined in: lib/types.ts:530

Eine kurze Zusammenfassung oder Beschreibung des Projekts.

_details

| private ***_details***: string

Defined in: lib/types.ts:542

Zusätzliche, detaillierte Projektinformationen.

_endDate

```
| private _endDate: string
```

Defined in: [lib/types.ts:534](#)

Das Enddatum des Projekts (Format: YYYY-MM-DD).

_location

```
| private _location: string
```

Defined in: [lib/types.ts:538](#)

Der geografische oder organisatorische Ort des Projekts.

_members

```
| private _members: ProjectMember\[\]
```

Defined in: [lib/types.ts:544](#)

Liste der dem Projekt zugewiesenen Mitglieder.

_startDate

```
| private _startDate: string
```

Defined in: [lib/types.ts:532](#)

Das Startdatum des Projekts (Format: YYYY-MM-DD).

_state

```
| private _state: string
```

Defined in: [lib/types.ts:536](#)

Der aktuelle Projektstatus (z. B. 'Ideen-Phase').

_title

```
| private _title: string
```

Defined in: [lib/types.ts:528](#)

Der Titel des Projekts.

_websiteUrl

| private **_websiteUrl**: string

Defined in: [lib/types.ts:540](#)

Optionale URL zur Projekt-Website.

Accessors

description

Get Signature

| **get description**(): string

Defined in: [lib/types.ts:576](#)

Returns

string

Set Signature

| **set description**(value): void

Defined in: [lib/types.ts:580](#)

Parameters

value

string

Returns

void

details

Get Signature

| **get details():** string

Defined in: [lib/types.ts:624](#)

Returns

string

Set Signature

| **set details(value):** void

Defined in: [lib/types.ts:628](#)

Parameters

value

string

Returns

void

endDate

Get Signature

| **get endDate():** string

Defined in: [lib/types.ts:592](#)

Returns

string

Set Signature

| **set endDate**(value): void

Defined in: [lib/types.ts:596](#)

Parameters

value

string

Returns

void

location

Get Signature

| **get location**(): string

Defined in: [lib/types.ts:608](#)

Returns

string

Set Signature

| **set location**(value): void

Defined in: [lib/types.ts:612](#)

Parameters

value

string

Returns

void

members

Get Signature

```
| get members(): ProjectMember\[\]
```

Defined in: [lib/types.ts:632](#)

Returns

[ProjectMember\[\]](#)

Set Signature

```
| set members(value): void
```

Defined in: [lib/types.ts:636](#)

Parameters

value

[ProjectMember\[\]](#)

Returns

void

startDate

Get Signature

| **get startDate():** string

Defined in: [lib/types.ts:584](#)

Returns

string

Set Signature

| **set startDate(value):** void

Defined in: [lib/types.ts:588](#)

Parameters

value

string

Returns

void

state

Get Signature

| **get state():** string

Defined in: [lib/types.ts:600](#)

Returns

string

Set Signature

| **set state**(value): void

Defined in: [lib/types.ts:604](#)

Parameters

value

string

Returns

void

title

Get Signature

| **get title**(): string

Defined in: [lib/types.ts:568](#)

Returns

string

Set Signature

| **set title**(value): void

Defined in: [lib/types.ts:572](#)

Parameters

value

string

Returns

void

websiteUrl

Get Signature

| **get websiteUrl():** string

Defined in: [lib/types.ts:616](#)

Returns

string

Set Signature

| **set websiteUrl(value):** void

Defined in: [lib/types.ts:620](#)

Parameters

value

string

Returns

void

Organization

Defined in: [lib/types.ts:1033](#)

Repräsentiert eine Organisation. * mermaid classDiagram class Organization { +number id +string name }

Constructors

Constructor

| **new Organization**(id, name): [Organization](#)

Defined in: [lib/types.ts:1039](#)

Parameters

id

number

name

string

Returns

[Organization](#)

Properties

_id

| private **_id**: number

Defined in: [lib/types.ts:1035](#)

Eindeutige ID der Organisation.

_name

| private **_name**: string

Defined in: [lib/types.ts:1037](#)

Der Name der Organisation.

Accessors

id

Get Signature

| **get id**(): number

Defined in: [lib/types.ts:1044](#)

Returns

number

Set Signature

| **set id**(value): void

Defined in: [lib/types.ts:1048](#)

Parameters

value

number

Returns

void

name

Get Signature

| **get name**() : string

Defined in: [lib/types.ts:1052](#)

Returns

string

Set Signature

| **set name**(value): void

Defined in: [lib/types.ts:1056](#)

Parameters

value

string

Returns

void

ProblemItem

Defined in: [lib/types.ts:251](#)

Repräsentiert eine Herausforderung oder ein Problem im System. *

```
mermaid classDiagram class ProblemItem { +number id +number problem_id +string title +string tags +string category +string status +string statusColor +string description +string summary +string impact +string stakeholders +string nextSteps }
```

Constructors

Constructor

```
new ProblemItem(title, category, status, statusColor, id?,  
problem_id?, tags?, description?, summary?, impact?,  
stakeholders?, nextSteps?): ProblemItem
```

Defined in: [lib/types.ts:277](#)

Parameters

title

string

category

string

status

"Ungelöst" | "In Bearbeitung" | "Gelöst"

statusColor

"amber" | "green" | "slate"

id?

number

problem_id?

number

tags?

string

description?

string

summary?

string

impact?

string

stakeholders?

string

nextSteps?

string

Returns

ProblemItem

Properties

_category

| private **_category**: string

Defined in: [lib/types.ts:261](#)

Die übergeordnete Kategorie des Problems.

_description?

| private optional ***_description?***: string

Defined in: [lib/types.ts:267](#)

Eine detaillierte Beschreibung des Problems oder kurze Zusammenfassung.

_id?

| private optional ***_id?***: number

Defined in: [lib/types.ts:253](#)

Eindeutige ID des Problems (Primärschlüssel).

_impact?

| private optional ***_impact?***: string

Defined in: [lib/types.ts:271](#)

Die Auswirkungen oder Konsequenzen, die das Problem verursacht.

_nextSteps?

| private optional ***_nextSteps?***: string

Defined in: [lib/types.ts:275](#)

Die nächsten geplanten Schritte zur Lösung des Problems.

_problem_id?

| private optional ***_problem_id?***: number

Defined in: [lib/types.ts:255](#)

Alternativer Feldname für die Problem-ID.

_stakeholders?

| private optional ***_stakeholders?***: string

Defined in: [lib/types.ts:273](#)

Die betroffenen Personen, Abteilungen oder Stakeholder.

_status

```
| private _status: "Ungelöst" | "In Bearbeitung" | "Gelöst"
```

Defined in: [lib/types.ts:263](#)

Der aktuelle Bearbeitungsstatus.

_statusColor

```
| private _statusColor: "amber" | "green" | "slate"
```

Defined in: [lib/types.ts:265](#)

Die dem Status zugewiesene UI-Farbe.

_summary?

```
| private optional _summary?: string
```

Defined in: [lib/types.ts:269](#)

Alternativer Feldname für Beschreibung.

_tags?

```
| private optional _tags?: string
```

Defined in: [lib/types.ts:259](#)

Kommagetrennte oder formatierte Tags/Schlagworte zur Verschlagwortung.

_title

```
| private _title: string
```

Defined in: [lib/types.ts:257](#)

Der Titel oder die Überschrift des Problems.

Accessors

category

Get Signature

| **get category**(): string

Defined in: [lib/types.ts:337](#)

Returns

string

Set Signature

| **set category**(value): void

Defined in: [lib/types.ts:341](#)

Parameters

value

string

Returns

void

description

Get Signature

| **get description**(): string | undefined

Defined in: [lib/types.ts:361](#)

Returns

string | undefined

Set Signature

| **set description**(value): void

Defined in: [lib/types.ts:365](#)

Parameters

value

string | undefined

Returns

void

id

Get Signature

| **get id**(): number | undefined

Defined in: [lib/types.ts:305](#)

Returns

number | undefined

Set Signature

| **set id**(value): void

Defined in: [lib/types.ts:309](#)

Parameters

value

number | undefined

Returns

void

impact

Get Signature

| **get *impact***(): string | undefined

Defined in: [lib/types.ts:377](#)

Returns

string | undefined

Set Signature

| **set *impact***(value): void

Defined in: [lib/types.ts:381](#)

Parameters

value

string | undefined

Returns

void

nextSteps

Get Signature

| **get nextSteps():** string | undefined

Defined in: [lib/types.ts:393](#)

Returns

string | undefined

Set Signature

| **set nextSteps(value):** void

Defined in: [lib/types.ts:397](#)

Parameters

value

string | undefined

Returns

void

problem_id

Get Signature

| **get problem_id():** number | undefined

Defined in: [lib/types.ts:313](#)

Returns

number | undefined

Set Signature

| **set problem_id**(value): void

Defined in: [lib/types.ts:317](#)

Parameters

value

number | undefined

Returns

void

stakeholders

Get Signature

| **get stakeholders**(): string | undefined

Defined in: [lib/types.ts:385](#)

Returns

string | undefined

Set Signature

| **set stakeholders**(value): void

Defined in: [lib/types.ts:389](#)

Parameters

value

string | undefined

Returns

void

status

Get Signature

| **get status():** "Ungelöst" | "In Bearbeitung" | "Gelöst"

Defined in: [lib/types.ts:345](#)

Returns

"Ungelöst" | "In Bearbeitung" | "Gelöst"

Set Signature

| **set status(value):** void

Defined in: [lib/types.ts:349](#)

Parameters

value

"Ungelöst" | "In Bearbeitung" | "Gelöst"

Returns

void

statusColor

Get Signature

| **get statusColor**(): "amber" | "green" | "slate"

Defined in: [lib/types.ts:353](#)

Returns

"amber" | "green" | "slate"

Set Signature

| **set statusColor**(value): void

Defined in: [lib/types.ts:357](#)

Parameters

value

"amber" | "green" | "slate"

Returns

void

summary

Get Signature

| **get summary**(): string | undefined

Defined in: [lib/types.ts:369](#)

Returns

string | undefined

Set Signature

| **set summary**(value): void

Defined in: [lib/types.ts:373](#)

Parameters

value

string | undefined

Returns

void

tags

Get Signature

| **get tags**(): string | undefined

Defined in: [lib/types.ts:329](#)

Returns

string | undefined

Set Signature

| **set tags**(value): void

Defined in: [lib/types.ts:333](#)

Parameters

value

string | undefined

Returns

void

title

Get Signature

| **get title():** string

Defined in: [lib/types.ts:321](#)

Returns

string

Set Signature

| **set title(value):** void

Defined in: [lib/types.ts:325](#)

Parameters

value

string

Returns

void

Project

Defined in: [lib/types.ts:78](#)

Repräsentiert ein registriertes (Smart-City-)Projekt. * mermaid

```
classDiagram
class Project {
+string id
+string title
+string stage
+string topics
+string location
+string[] actors
+string value
}
```

Constructors

Constructor

```
new Project(id, title, stage?, topics?, location?, actors?,  
value?): Project
```

Defined in: [lib/types.ts:94](#)

Parameters

id

string

title

string

stage?

string

topics?

string

location?

string

actors?

string[]

value?

string

Returns

Project

Properties

_actors?

```
| private optional _actors?: string[]
```

Defined in: [lib/types.ts:90](#)

Eine Liste der Namen aller beteiligten Akteure/Experten.

_id

```
| private _id: string
```

Defined in: [lib/types.ts:80](#)

Eindeutige ID des Projekts.

_location?

```
| private optional _location?: string
```

Defined in: [lib/types.ts:88](#)

Der geografische Ort oder Stadtteil des Projekts.

_stage?

```
| private optional _stage?: string
```

Defined in: [lib/types.ts:84](#)

Die aktuelle Projektphase oder Themen (z. B. “Test-Phase”, “Technik”).

_title

| private **_title**: string

Defined in: [lib/types.ts:82](#)

Der Projekttitel.

_topics?

| private optional **_topics?**: string

Defined in: [lib/types.ts:86](#)

Alternativer Feldname für Themen oder Standort.

_value?

| private optional **_value?**: string

Defined in: [lib/types.ts:92](#)

Ein dynamischer UI-Anzeigewert (Teaser-Text, URL oder formatiertes Update-Datum).

Accessors

actors

Get Signature

| **get actors**(): string[] | undefined

Defined in: [lib/types.ts:152](#)

Returns

string[] | undefined

Set Signature

| **set actors**(value): void

Defined in: [lib/types.ts:156](#)

Parameters

value

string[] | undefined

Returns

void

id

Get Signature

| **get id**(): string

Defined in: [lib/types.ts:112](#)

Returns

string

Set Signature

| **set id**(value): void

Defined in: [lib/types.ts:116](#)

Parameters

value

string

Returns

void

location

Get Signature

| **get location**(): string | undefined

Defined in: [lib/types.ts:144](#)

Returns

string | undefined

Set Signature

| **set location**(value): void

Defined in: [lib/types.ts:148](#)

Parameters

value

string | undefined

Returns

void

stage

Get Signature

| **get stage()**: string | undefined

Defined in: [lib/types.ts:128](#)

Returns

string | undefined

Set Signature

| **set stage(value)**: void

Defined in: [lib/types.ts:132](#)

Parameters

value

string | undefined

Returns

void

title

Get Signature

| **get title()**: string

Defined in: [lib/types.ts:120](#)

Returns

string

Set Signature

| **set title**(value): void

Defined in: [lib/types.ts:124](#)

Parameters

value

string

Returns

void

topics

Get Signature

| **get topics**(): string | undefined

Defined in: [lib/types.ts:136](#)

Returns

string | undefined

Set Signature

| **set topics**(value): void

Defined in: [lib/types.ts:140](#)

Parameters

value

string | undefined

Returns

void

value

Get Signature

| **get value**(): string | undefined

Defined in: [lib/types.ts:160](#)

Returns

string | undefined

Set Signature

| **set value**(value): void

Defined in: [lib/types.ts:164](#)

Parameters

value

string | undefined

Returns

void

ProjectDetails

Defined in: [lib/types.ts:662](#)

Struktur für die vollständigen Details eines Projekts. * mermaid
classDiagram direction LR ProjectDetails "1" --* "many"
ProjectMember : besitzt class ProjectDetails { +string id +string
title +string description +string startDate +string endDate
+string state +string project_state +string location +string
websiteUrl +string details +ProjectMember[] experts }

Constructors

Constructor

```
new ProjectDetails(id, title, description, startDate, endDate,  
state, location, websiteUrl, details, experts, project_state?):  
ProjectDetails
```

Defined in: [lib/types.ts:686](#)

Parameters

id

string

title

string

description

string

startDate

string

endDate

string

state

string

location

string

websiteUrl

string

details

string

experts

ProjectMember[]

project_state?

string

Returns

ProjectDetails

Properties

_description

| private **_description**: string

Defined in: [lib/types.ts:668](#)

Kurze Zusammenfassung oder Beschreibung des Projekts.

_details

```
| private _details: string
```

Defined in: [lib/types.ts:682](#)

Zusätzliche, detaillierte Projektbeschreibungen oder Notizen.

_endDate

```
| private _endDate: string
```

Defined in: [lib/types.ts:672](#)

Enddatum des Projekts (Format: YYYY-MM-DD).

_experts

```
| private _experts: ProjectMember[]
```

Defined in: [lib/types.ts:684](#)

Liste aller dem Projekt zugewiesenen Experten.

_id

```
| private _id: string
```

Defined in: [lib/types.ts:664](#)

Eindeutige ID des Projekts.

_location

```
| private _location: string
```

Defined in: [lib/types.ts:678](#)

Geografischer oder organisatorischer Standort des Projekts.

_project_state?

| private optional **_project_state?**: string

Defined in: [lib/types.ts:676](#)

Alternativer oder datenbankspezifischer Projektstatus.

_startDate

| private **_startDate**: string

Defined in: [lib/types.ts:670](#)

Startdatum des Projekts (Format: YYYY-MM-DD).

_state

| private **_state**: string

Defined in: [lib/types.ts:674](#)

Der aktuelle Projektstatus (z. B. 'Ideen-Phase').

_title

| private **_title**: string

Defined in: [lib/types.ts:666](#)

Der Projekttitel.

_websiteUrl

| private **_websiteUrl**: string

Defined in: [lib/types.ts:680](#)

Optionale Projekt-Website-URL.

Accessors

description

Get Signature

| **get description**(): string

Defined in: [lib/types.ts:728](#)

Returns

string

Set Signature

| **set description**(value): void

Defined in: [lib/types.ts:732](#)

Parameters

value

string

Returns

void

details

Get Signature

| **get details**(): string

Defined in: [lib/types.ts:784](#)

Returns

string

Set Signature

| **set details**(value): void

Defined in: [lib/types.ts:788](#)

Parameters

value

string

Returns

void

endDate

Get Signature

| **get endDate**(): string

Defined in: [lib/types.ts:744](#)

Returns

string

Set Signature

| **set endDate**(value): void

Defined in: [lib/types.ts:748](#)

Parameters

value

string

Returns

void

experts

Get Signature

| **get experts():** ProjectMember[]

Defined in: lib/types.ts:792

Returns

ProjectMember[]

Set Signature

| **set experts(value):** void

Defined in: lib/types.ts:796

Parameters

value

ProjectMember[]

Returns

void

id

Get Signature

| **get id():** string

Defined in: [lib/types.ts:712](#)

Returns

string

Set Signature

| **set id(value):** void

Defined in: [lib/types.ts:716](#)

Parameters

value

string

Returns

void

location

Get Signature

| **get location():** string

Defined in: [lib/types.ts:768](#)

Returns

string

Set Signature

| **set location**(value): void

Defined in: [lib/types.ts:772](#)

Parameters

value

string

Returns

void

project_state

Get Signature

| **get project_state**(): string | undefined

Defined in: [lib/types.ts:760](#)

Returns

string | undefined

Set Signature

| **set project_state**(value): void

Defined in: [lib/types.ts:764](#)

Parameters

value

string | undefined

Returns

void

startDate

Get Signature

get startDate(): string

Defined in: [lib/types.ts:736](#)

Returns

string

Set Signature

set startDate(value): void

Defined in: [lib/types.ts:740](#)

Parameters

value

string

Returns

void

state

Get Signature

| **get state()**: string

Defined in: [lib/types.ts:752](#)

Returns

string

Set Signature

| **set state(value)**: void

Defined in: [lib/types.ts:756](#)

Parameters

value

string

Returns

void

title

Get Signature

| **get title()**: string

Defined in: [lib/types.ts:720](#)

Returns

string

Set Signature

| **set title**(value): void

Defined in: [lib/types.ts:724](#)

Parameters

value

string

Returns

void

websiteUrl

Get Signature

| **get websiteUrl**(): string

Defined in: [lib/types.ts:776](#)

Returns

string

Set Signature

| **set websiteUrl**(value): void

Defined in: [lib/types.ts:780](#)

Parameters

value

string

Returns

void

ProjectMember

Defined in: [lib/types.ts:414](#)

Repräsentiert ein Mitglied innerhalb eines Projekts inklusive seiner RACI-Rolle.*

```
mermaid classDiagram class ProjectMember { +string id +number expertId +string name +string role }
```

Constructors

Constructor

`new ProjectMember(id, name, role, expertId?): ProjectMember`

Defined in: [lib/types.ts:429](#)

Parameters

id

string

name

string

role

"R" | "A" | "C" | "I"

expertId?

number

Returns

ProjectMember

Properties

_expertId?

```
| private optional _expertId?: number
```

Defined in: lib/types.ts:418

Die echte ID des Experten aus der Datenbank (optional).

_id

```
| private _id: string
```

Defined in: lib/types.ts:416

Eindeutige temporäre ID für das UI-Mapping (z. B. generiert über `Date.now()`).

_name

```
| private _name: string
```

Defined in: lib/types.ts:420

Der Name des Experten.

_role

```
| private _role: "R" | "A" | "C" | "I"
```

Defined in: [lib/types.ts:427](#)

Die RACI-Rolle des Mitglieds im Projekt: - R: Responsible (Durchführungsverantwortlich) - A: Accountable (Kosten-/ Gesamtverantwortlich) - C: Consulted (Fachlich beratend) - I: Informed (Zu informieren)

Accessors

expertId

Get Signature

| **get expertId()**: number | undefined

Defined in: [lib/types.ts:444](#)

Returns

number | undefined

Set Signature

| **set expertId(value)**: void

Defined in: [lib/types.ts:448](#)

Parameters

value

number | undefined

Returns

void

id

Get Signature

| **get id():** string

Defined in: [lib/types.ts:436](#)

Returns

string

Set Signature

| **set id(value):** void

Defined in: [lib/types.ts:440](#)

Parameters

value

string

Returns

void

name

Get Signature

| **get name():** string

Defined in: [lib/types.ts:452](#)

Returns

string

Set Signature

| **set name**(value): void

Defined in: [lib/types.ts:456](#)

Parameters

value

string

Returns

void

role

Get Signature

| **get role**(): "R" | "A" | "C" | "I"

Defined in: [lib/types.ts:460](#)

Returns

"R" | "A" | "C" | "I"

Set Signature

| **set role**(value): void

Defined in: [lib/types.ts:464](#)

Parameters

value

"R" | "A" | "C" | "I"

Returns

void

Interfaces

AddExpertViewProps

Defined in: [lib/types.ts:814](#)

Eigenschaften (Props) für eine Komponente zum Hinzufügen oder Bearbeiten eines Experten. Callbacks sind optional, da die Komponente alternativ integrierte Fallback-Navigation besitzt. * mermaid

```
classDiagram direction LR AddExpertViewProps ..> ExpertFormData :  
nutzt class AddExpertViewProps { +onSave(formData: ExpertFormData)  
void +onCancel() void }
```

Properties

onCancel?

| optional **onCancel?**: () => void

Defined in: [lib/types.ts:823](#)

Callback-Funktion, um den Vorgang abubrechen und zur vorherigen Ansicht zurückzukehren.

Returns

void

onSave?

optional **onSave?**: (formData) => void

Defined in: [lib/types.ts:819](#)

Callback-Funktion, die aufgerufen wird, wenn die Expertendaten erfolgreich validiert und gespeichert wurden.

Parameters

formData

[ExpertFormData](#)

Die eingegebenen Formulardaten des Experten.

Returns

void

Documentation

Documentation

Modules

- [app/api/challenges/add/route](#)
- [app/api/challenges/delete/route](#)
- [app/api/challenges/list/route](#)
- [app/api/sandbox/add/route](#)
- [app/api/sandbox/list/route](#)
- [app/api/sandbox/update/route](#)
- [app/api/users/create/route](#)
- [app/api/users/delete/route](#)
- [app/api/users/list/route](#)
- [app/api/users/update/route](#)
- [app/challenge/add/page](#)

- [app/challenge/page](#)
- [app/components/Agentbar](#)
- [app/components/Sidebar](#)
- [app/components/Topbar](#)
- [app/layout](#)
- [app/page](#)
- [app/relations/add/page](#)
- [app/relations/page](#)
- [app/relations/Person](#)
- [app/relations/update/page](#)
- [app/sandbox/add/page](#)
- [app/sandbox/page](#)
- [app/sandbox/project/details/page](#)
- [app/search/page](#)
- [lib/db](#)
- [lib/types](#)
- [next.config](#)
- [prisma.config](#)

[Documentation](#)

[Documentation](#) / [next.config](#)

next.config

Variables

default

```
| const default: NextConfig = {}
```

Defined in: [next.config.ts:3](#)

[Documentation](#)

[Documentation](#) / [prisma.config](#)

prisma.config

Variables

default

| **default:** PrismaConfigInternal

Defined in: [prisma.config.ts:6](#)